НАСТРОЙКА ОКРУЖЕНИЯ

Параметр	Значение
Тип документа	План и регламент подготовки рабочих средств, доступов, инфраструктуры и демоданных
Основание	Устав проекта, концепт, ИСР, бэклог, расписание, план итераций
Дата составления	26.12.2026
Принцип фиксации	Ниже описаны требуемые действия и критерии их проверки, а не отчёт об уже выполненной установке

1. Назначение и границы работ

Цель подготовки окружения — обеспечить возможность всей команде воспроизводимо запускать клиент-серверное веб-приложение, совместно изменять исходный код, собирать и тестировать его, использовать единую структуру базы данных и показывать заказчику достоверные демонстрационные сценарии. Подготовка выполняется до начала соответствующих функциональных работ и поддерживается при переходе между итерациями.

Предмет проекта — веб-приложение строительной компании. Система должна включать каталог проектов, конфигуратор дома, калькулятор ремонта, форму заявки, выгрузку PDF-сметы и базовую админку. Отзывы, FAQ, портфолио «до/после» и расширенная админка в базовую поставку не входят.

Документ охватывает учётные записи, инструменты разработки, Git-репозиторий, контейнеризацию, конфигурации, PostgreSQL, автоматическую сборку и проверки, тестовый стенд, демонстрационные данные, ответственность, контроль готовности и риски. Конкретные поставщики сервисов, адреса репозиториев и фактическое наличие доступов в исходных файлах не зафиксированы; они обозначены как решения, требующие подтверждения.

2. Место подготовки окружения в плане проекта

Итерация 1 (Дни 1–7) отведена анализу, архитектуре, фиксации правил и настройке инфраструктуры. Рабочего приложения в ней в полном объеме не предполагается. После согласования архитектуры можно переходить к среде разработки и последующим функциональным модулям.

Пакет ИСР	Когда и объём	Предпосылки / ожидаемый результат
D1. Настройка среды	Итерация	После согласования архитектуры.

Пакет ИСР	Когда и объём	Предпосылки / ожидаемый результат
разработки и CI/CD	1 (Дни 1–2).	Контейнеры для локальной разработки, воспроизводимая сборка, автоматические линтеры и модульные тесты, доступный тестовый стенд.
D2. Создание демонстрационных данных	Итерация 1 (Дни 3–5).	После реализации базовых модулей инфраструктуры (каталог, конфигуратор). Набор воспроизводимых данных-заглушек для демонстрации и тестирования.
D3. Развёртывание production-окружения	Итерация 3 (Дни 19–21).	Отдельная поздняя задача; не считать полноценный production частью начальной настройки.

Установленные даты — плановые контрольные сроки; фактическое выполнение по доске задач не подтверждено.

3. Организационные учётные записи и права доступа

Роли руководителя проекта, backend-разработчика, frontend-разработчика и QA/дизайнера отражены в реестре заинтересованных сторон; при выдаче прав необходимо учитывать их разные полномочия.

Руководитель проекта формирует перечень участников и проверяет получение доступа к рабочим системам. Доступы выдаются по роли, без общих личных паролей. При необходимости включается двухфакторная аутентификация. Для внешних участников (заказчик) предоставляются только необходимые права просмотра, демонстрации или приёмки.

Система / учётная запись	Что подготовить	Как подтвердить готовность
Таск-трекер (YouGile/Trello)	Проверить или создать проект; пригласить 4 участников; завести разделы/задачи ИСР, статусы, приоритеты, ответственных и итерации.	Каждый участник видит свой бэклог и может менять статус назначенных задач; руководитель видит все работы.
Хостинг исходного кода Git	Согласовать площадку (GitHub/GitLab); создать репозиторий и роли Owner/Maintainer/Developer; дать членам команды доступ.	Все разработчики успешно клонируют репозиторий, создают рабочие ветки и открывают запросы на слияние.
Хранилище документов	Определить общее расположение устава, концепта, бэклога, макетов, схем API и инструкций.	Участники видят актуальные версии; есть единые правила именования и изменения.

Система / учётная запись	Что подготовить	Как подтвердить готовность
CI/CD и реестр образов	Выдать доступ к выбранному CI-сервису (например, GitHub Actions), секретам проекта и контейнерному реестру.	Участник с нужными правами может запустить pipeline и просмотреть отчёт.
Тестовый стенд и БД	Выдать разработчику и QA сетевой доступ; администратору/заказчику — тестовые аккаунты приложения.	Доступность интерфейса и API проверена с рабочих устройств команды.

Порядок оформления доступа

- Руководитель проекта составляет матрицу: сервис → участник → роль → дата выдачи → подтверждение входа.
- Каждый участник подтверждает регистрацию через личную почту и меняет временный пароль; права выдаются по минимальной необходимости.
- Назначаются резервные владельцы ресурсов и порядок отзыва прав при выходе из команды.
- Реквизиты тестовых пользователей приложения хранятся отдельно от паролей внешних систем; секреты не публикуются в Git и проектной документации.

4. Установка программного обеспечения

Базовый технологический стек определён Уставом: Node.js (или Python), JavaScript/TypeScript, Vite, React/Vue, PostgreSQL, Docker. Для совместной работы необходим Git. Остальные версии и инструменты выбираются и фиксируются командой после согласования архитектуры; приведённые ниже позиции не следует считать уже утверждёнными.

Компонент	Обязательное действие / назначение	Что уточнить
Node.js (или Python)	Установить среду выполнения и пакетный менеджер; проверить node -v или python --version.	Единая версия среды.
Фреймворки (React/Vue + Vite)	Инициализировать frontend на совместимых зависимостях; зафиксировать версии в файле сборки.	Набор UI-библиотек (MUI, Tailwind).
PostgreSQL	Обеспечить базу локально через контейнер; задать отдельные	Версия PostgreSQL и правила миграций.

Компонент	Обязательное действие / назначение	Что уточнить
	роли/схемы для приложения и тестов.	
Docker + Compose	Обеспечить запуск сервисов одной командой; проверить взаимодействие контейнеров.	Поддерживаемые ОС, порты, лимиты ресурсов.
Git, IDE, средства проверки API	Настроить Git и редакторы/IDE по ролям; предусмотреть запуск HTTP-запросов и тестов.	Конкретные IDE, Postman/альтернатива.

Минимальная проверка рабочей станции

В терминале проверяются команды git --version, node --version, docker --version и docker compose version. Запись результата проверки и обнаруженных проблем добавляется в карточку D1. Проверка отдельного клиента PostgreSQL нужна лишь при его использовании вне контейнеров.

5. Организация Git-репозитория

До начала функциональной разработки нужно создать единое место хранения кода с понятной структурой. Исходные документы не закрепляют Git-платформу, поэтому ниже приведён предлагаемый организационный вариант для согласования.

Элемент	Предлагаемая настройка
Каталоги	frontend/ (Vite/React), backend/ (API), infra/ (Compose, CI), docs/ (схемы, API, решения), scripts/ (демоданные, утилиты).
Ветки	main — стабильная версия, короткие рабочие ветки по ID задачи (feature/название); состав и правила веток закрепляются до первого merge.
Правила работы	Pull/Merge Request, хотя бы одна проверка другим участником для критичного кода, запрет прямой записи в защищённую ветку, зеленая автоматическая сборка перед слиянием.
Служебные файлы	README.md, gitignore, .env.example, конфигурация CI, пример команд запуска, инструкция заполнения БД, шаблон описания MR.

Начальный коммит должен содержать каркас backend и frontend, описание запуска, конфигурации контейнеров и минимум один автоматически проверяемый пример теста. Репозиторий не должен содержать персональные данные заказчика, секреты, реальные пароли или базы данных с чувствительной информацией.

6. Локальное окружение и конфигурации

6.1. Контейнеры и взаимодействие частей системы

- Подготовить Docker Compose для PostgreSQL и backend (или запускать Vite на хосте при документированном сетевом адресе API). Для каждого сервиса описать старт, порты, зависимости и проверку готовности.
- Обеспечить, чтобы backend обращался к БД по внутреннему имени сервиса, а браузер — к API по корректному внешнему адресу. Зафиксировать настройки CORS и доступные origins.
- Вынести параметры подключения, секреты, режимы разработки и тестирования в переменные окружения; опубликовать только безопасный шаблон .env.example.
- Добавить простой способ сброса локальной БД и повторного развёртывания демонстрационного набора; избегать уничтожения данных без явного предупреждения.
- Описать два маршрута запуска: «с нуля» и «обновление текущего окружения», а также остановку и диагностику типовых ошибок.

6.2. Подготовка базы данных

Схема PostgreSQL должна отражать пользователей, проекты домов, конфигурации, детали ремонта и заявки. Правила расчета цен критичны — они должны быть вынесены в отдельный модуль (движок цен), чтобы их можно было менять без переписывания всего бэкенда.

- Определить механизм версионирования схемы БД (например, Prisma, TypeORM, Alembic). Миграции должны применяться повторяемо на новом окружении.
- Разделить базы/профили для локальной разработки и автоматических тестов; не выполнять демонстрационные сбросы на production.
- Проверить создание таблиц, внешние ключи, индексы ключевых выборок и серверное отклонение недопустимых действий; бизнес-правила не должны зависеть только от frontend.

7. Автоматическая сборка, проверки и тестовый стенд

Для D1 в исходной оценке прямо предусмотрены автоматическая сборка, запуск линтеров и модульных тестов. CI/CD подготавливается в объёме, достаточном для интеграции и раннего выявления дефектов, а выпуск production отнесён к D3.

Шаг	Проверка	Условие прохождения
1. Загрузка кода	Получение определённой версии	Сборка запускается с

Шаг	Проверка	Условие прохождения
	backend/frontend и зависимостей.	чистого окружения.
2. Статические проверки	Линтеры (ESLint/Prettier), форматирование и базовые проверки проекта.	Ошибки проверок блокируют merge по согласованным правилам.
3. Модульные тесты	Критичные сервисные проверки: логика калькулятора ремонта, валидация конфигуратора, расчет цен.	Тесты запускаются автоматически, результаты видны команде.
4. Сборка артефактов	Backend-приложение, frontend-сборка и при необходимости Docker-образы.	Артефакты собираются без ручных изменений конфигурации.
5. Smoke-проверка	Старт приложения и соединение с PostgreSQL, доступность API и интерфейса.	Простой проверочный сценарий выполняется на тестовом стенде.

Тестовый стенд используется для непрерывной проверки функциональности и демонстраций. Его адреса, порядок обновления, права доступа, способ восстановления базы и ответственность за доступность документируются отдельно. К концу 1-й недели должно быть возможно показать работающую сборку и БД.

8. Подготовка демонстрационных данных (D2)

Демоданные нужны для проверки инфраструктуры каталога, конфигуратора, авторизации и последующего бронирования. Заказчик предоставляет реальный контент не сразу (дедлайн 14-й день), поэтому команда работает на заглушках.

8.1. Рекомендуемый состав тестового набора

Набор	Что должно быть	Для какой проверки
Каталог проектов	2-3 условных проекта домов; разная площадь, этажность, базовые цены.	Создание объектов, иерархия, фильтры.
Конфигуратор	Набор опций для выбора (материалы, планировка), влияющих на цену.	Валидация характеристик и работа движка цен.
Пользователи	1 администратор и 2 тестовых клиента.	Авторизация, ролевые запреты.
Тарифы ремонта	Базовые расценки на работы и материалы для калькулятора.	Проверка математики расчета.
Заявки	Успешно созданные, завершенные и отмененные записи.	Статусы, поиск, выгрузка PDF.

Набор	Что должно быть	Для какой проверки
Отрицательные случаи	Некорректные размеры, отсутствие опций, пустые поля.	Серверное отклонение запрещенных операций.

8.2. Воспроизводимое создание и очистка

- Создать версионируемый сид (SQL или программный загрузчик) и понятный сценарий применения только к dev/test БД.
- Не использовать реальные персональные данные: имена и контакты должны быть вымышленными.
- Задавать стабильные идентификаторы и предсказуемые связи, чтобы QA мог повторить каждый тест; сценарии ссылать на ID соответствующей записи.
- Сделать проверку после загрузки: ожидаемое количество проектов, пользователей, заявок и отсутствие нарушений ссылочной целостности.

8.3. Набор контрольных ситуаций

ID	Сценарий	Ожидаемый результат
DEMO-01	Администратор создает проект и опции.	Объект доступен в API, а затем — в интерфейсе.
DEMO-02	Пользователь выбирает опции в конфигураторе.	Цена пересчитывается мгновенно и корректно.
DEMO-03	Пользователь отправляет заявку.	Заявка сохраняется в БД, генерируется PDF.
DEMO-04	Ввод некорректных размеров/данных.	Сервер отклоняет действие, выдается ошибка.
DEMO-05	Запрос к админке без прав.	Доступ запрещен (403).

Ожидаемые ограничения вытекают из устава и критериев приёмки бэклога. Нельзя выдавать выбранные тестовые значения за требования заказчика.

---

9. Распределение ответственности

Роль	Зона ответственности при подготовке	Проверяемый результат
Руководитель проекта	Организует выдачу доступов, общую доску, контроль сроков (21 день), согласование инфраструктуры.	Матрица доступов, задачи D1/D2, контроль готовности.
Backend-разработчик	Ведущий по D1: Node.js/Python, PostgreSQL, Docker, сборка и CI/CD; обеспечивает движок цен и автоматические	Работающий backend, БД, Compose,

Роль	Зона ответственности при подготовке	Проверяемый результат
	API.	проверки.
Frontend-разработчик	Подготавливает Vite/React/Vue, конфигурацию API, локальную сборку и макеты интерфейса.	Frontend запускается и обращается к API на стенде.
Дизайнер / QA	Проверяет чистый запуск, воспроизводимость, позитивные/негативные сценарии и сохранение результатов.	Чек-листы, тест-кейсы, протокол первичных проверок.

---

10. Проверка готовности к непосредственной разработке

Окружение считается подготовленным только после фактической проверки перечисленных пунктов.

- [ ] Все участники получили и проверили права к задачам, исходному коду и общим проектным материалам.
- [ ] Архитектура и выбранные версии инструментов зафиксированы; Node.js, Git и Docker работают на нужных машинах.
- [ ] Исходный код доступен через Git; README позволяет новому участнику запустить проект без устных инструкций.
- [ ] Backend, frontend и PostgreSQL запускаются из чистого окружения, соединение между ними проверено.
- [ ] Схема БД разворачивается повторяемо; её версия и порядок миграций зафиксированы.
- [ ] Автоматическая сборка, линтеры и модульные тесты запускаются в CI и дают однозначный результат.
- [ ] Тестовый стенд доступен команде; известны адреса, тестовые роли и порядок восстановления.
- [ ] Внесён базовый набор демоданных D2; проверены количество записей и отрицательные сценарии.
- [ ] Список открытых архитектурных вопросов и расхождений документации передан руководителю проекта и заказчику.

11. Результаты, передаваемые по итогам подготовки

- **По D1:** ссылка на Git-репозиторий; согласованная структура кода; инструкции локального запуска; файлы контейнеризации; шаблон переменных окружения; работающий тестовый стенд; настроенная CI-сборка и журнал первого успешного pipeline.

- **По D2:** сид/скрипт генерации данных, описание объектов и тестовых аккаунтов, инструкция сброса и повторной загрузки, перечень эталонных сценариев, протокол проверки целостности.
- **Для управления:** матрица доступов; ответственные лица и открытые вопросы; запись демонстрации или протокол показа результата; ссылки на задачи и связанные изменения.
- **Для следующих этапов:** согласованные API-подход и модель БД, базовые тест-кейсы, описание ограничений, исключений и порядка внесения изменений.

12. Основные риски подготовки и меры

Риски и стратегии обработки перечислены в приложенном реестре рисков; ниже указаны непосредственно применимые к подготовке окружения действия.

Риск	Раннее действие при настройке
R-01: Сложная логика расчета цен	Вынести расчет в отдельный модуль (движок цен), покрыть тестами в CI до начала интеграции с фронтендом.
R-02: Задержка контента от заказчика	Работать на заглушках (фейковые фото, тексты) до 14-го дня; зафиксировать дедлайн передачи контента.
R-03: Жесткий срок (21 день)	Запустить тесты в CI на раннем этапе, сохранить негативные сценарии как обязательную часть DoD. Не тратить больше 2 дней на настройку D1.
R-07 / R-09: Отсутствие участников	Документировать запуск, назначить резервные доступы и роли, не хранить знания только на одном ПК.
R-13: Атаки и утечка учётных данных	Не хранить секреты в репозитории; разделить права; применять параметризованные запросы и защиту аутентификации.
R-14: Отказ сервиса и инфраструктуры	Документировать восстановление окружения; для production на следующем этапе предусмотреть бэкапы, мониторинг и SSL.

13. Источники проекта

Документ составлен по приложенным материалам. необходимые организационные предложения помечены словами «предлагается», «рекомендуется», «уточнить» или «согласовать».

1. Устав_проекта.pdf — Цель, состав, сроки, ограничения, роли, технологический стек.

2. Жизненный_цикл_проекта.docx — Гибридная модель, Agile, модель Киневин, факторы риска.
3. Бэклог.pdf — Итерации, D1/D2, критерии готовности.
4. Расписание.pdf — Пакеты ИСР, сроки (21 день), ресурсные ограничения.
5. Реестр_рисков.xlsx — Риски и рекомендуемые меры контроля. контроля.